Softmax

How Logits Become Probabilities

1 The Problem

The `lm_head` layer outputs **49,153 raw scores** (logits), one for each token in the vocabulary. These scores can be any number — positive, negative, large, small:

$$\text{logits} = [\underbrace{2.0}_{\text{token 0}}, \underbrace{5.0}_{\text{token 1}}, \underbrace{1.0}_{\text{token 2}}, \underbrace{-1.0}_{\text{token 3}}, \underbrace{0.5}_{\text{token 4}}, \ldots]$$

We need to turn these into **probabilities** (0% to 100%, summing to exactly 100%) so the model can pick the next token.

Analogy: exam scores to ranking

Imagine 5 students take an exam. Their raw scores are 2, 5, 1, −1, 0.5. You want to say "Student B has a 92% chance of being the best." Raw scores alone don't give you that — you need a formula to convert scores to percentages. That formula is **softmax**.

2 The Softmax Formula

For a vocabulary of V tokens (in SmolLM, $V = 49{,}153$):

$$P(\text{token}_i) = \frac{e^{\text{logit}_i}}{\sum_{j=0}^{V-1} e^{\text{logit}_j}}$$

where $e \approx 2.71828$ (Euler's number).

Key Point

Three guarantees of softmax:

1. Every probability is **positive** (even if the logit was negative, because $e^x > 0$ for all x).
2. All probabilities **sum to exactly 100%**.
3. The **highest logit** always gets the **highest probability**.

3 Step-by-Step Example

Let's use a tiny vocabulary of only 5 tokens.

Raw logits from lm_head

$$\text{logits} = [2.0,\ 5.0,\ 1.0,\ -1.0,\ 0.5]$$

Step 1: Raise e to the power of each logit

$$\begin{aligned} e^{2.0} &= 7.389 \\ e^{5.0} &= 148.413 \\ e^{1.0} &= 2.718 \\ e^{-1.0} &= 0.368 \\ e^{0.5} &= 1.649 \end{aligned}$$

Key Point

Notice that $e^{-1.0} = 0.368$ — it's *positive* even though the logit was negative. This is why softmax guarantees all probabilities are positive.

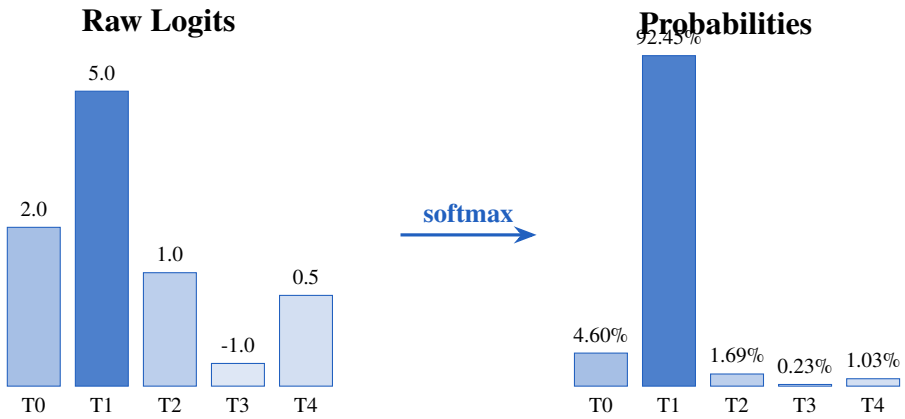
Step 2: Sum all the values

$$\text{total} = 7.389 + 148.413 + 2.718 + 0.368 + 1.649 = \mathbf{160.537}$$

Step 3: Divide each by the total

Token	Logit	e^{logit}	Calculation	Probability
0	2.0	7.389	7.389 / 160.537	**4.60**%
1	5.0	148.413	148.413 / 160.537	**92.45**%
2	1.0	2.718	2.718 / 160.537	**1.69**%
3	−1.0	0.368	0.368 / 160.537	**0.23**%
4	0.5	1.649	1.649 / 160.537	**1.03**%
			Total	**100.00**%

Visual: logits vs. probabilities



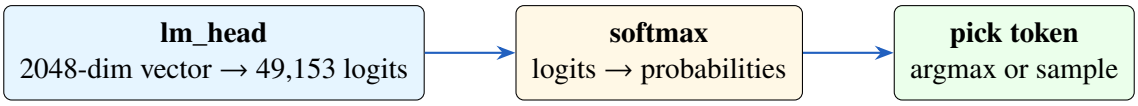
4 What Softmax Does (Intuition)

1. **Amplifies differences.** Logit 5.0 vs. 2.0 is only a 3-point gap, but after softmax it becomes 92% vs. 5%. The exponential e^x makes big logits *much* bigger relative to small ones.
2. **Makes negatives positive.** Logit -1.0 becomes $e^{-1.0} = 0.368$, which gives 0.23%. No token ever gets a negative probability.
3. **Normalises to 100%.** By dividing by the sum, all probabilities add up to exactly 1 (100%). This lets us treat the output as a proper probability distribution.

5 In the Real Model

In SmolLM, this happens with **49,153** logits instead of 5:

$$P(\text{token}_i) = \frac{e^{\text{logit}_i}}{\sum_{j=0}^{49152} e^{\text{logit}_j}}$$



In the Python code, this is one line:

```
probs = torch.nn.functional.softmax(last_logits.float(), dim=0)
```

It applies the formula above to all 49,153 logits at once, producing a probability for every token in the vocabulary.

6 Summary

What we learned

1. `lm_head` outputs **raw logits** — one score per vocabulary token (49,153 numbers).
2. Logits can be any number (positive or negative) and don't sum to 100%.
3. **Softmax** converts logits to probabilities using $P_i = e^{\text{logit}_i} / \sum e^{\text{logit}_j}$.
4. After softmax: all values are **positive** and **sum to exactly 100%**.
5. The model picks the next token by taking the **highest probability** (greedy) or **sampling** from the distribution.